

Operating Envelopes for Memristor Crossbar Non-Idealities: A Calibrated Simulation Study from Device Physics to In-Situ Learning

Jyotirmoy Ray
Wipro Research
Bengaluru, India
jyotirmoy.ray@wipro.com

Abstract—Analog memristor crossbars promise large energy savings for neural-network inference by performing matrix-vector multiplication in place, but their practical accuracy is governed by device and circuit non-idealities. We build a single, physically grounded compact model of an oxide valence-change cell, calibrate it to a published self-rectifying Ag_2O device, and scale it, without changing any equation, from one device up to a 64×64 crossbar and a small trained network. Using one consistent model we reproduce the pinched current-voltage hysteresis and its collapse with sweep frequency, quantify device-to-device variability across 4096 cells, solve the full wire-resistance network with modified nodal analysis, and model thermally activated retention loss. We then study three questions about how these effects interact with computation; all headline numbers are averaged over multiple seeds. First, closed-loop program-verify writing suppresses the accuracy loss from variability (a 64 to 10 classifier recovers from 88.3 to 96.5 percent over five seeds, matching a 96.5 percent software baseline). Second, finite wire resistance scales sharply with array size (the mean output error of an all-low-resistance array rises from 19 to 92 percent from 16×16 to 128×128 at 2.5 ohm per cell, and a realistic mixed-state map roughly halves these numbers). Third, wire resistance is a static, structured distortion that on-chip training can largely absorb: a two-spiral network deployed naively under IR drop sits at 63.5 percent and a cheap output calibration recovers only to 69.3 percent, but training with the IR-drop network in the forward pass recovers it to 92.4 percent, whereas thermally activated retention drift, being time varying, degrades a trained network to chance over time at elevated temperature and cannot be trained away. Under this model the three non-idealities tend to separate by remedy: variability is correctable at write time, IR drop is absorbable by training (and not by output calibration alone), and retention is only deferrable by rewriting. We fit the model to the measured figures in the device’s supplementary material (its conductance window, 100 states, and an activation energy $E_a = 1.45$ eV from its Arrhenius retention plot); on its 8×8 MNIST task the calibrated device adds only a 0.8-point accuracy penalty, matching an established hardware-aware toolkit (AIHWKit, 0.6 points) and the near-zero penalty the authors report. We then chart operating envelopes over array size, wire resistance, and temperature that delineate where each remedy is needed, and an energy model showing 18 to 41 TOPS/W (versus about 4 for a digital baseline) bounded by the same IR-drop limit. Finally we map a vision-based robotics steering network onto the array with full system realism (layer tiling, a $1\text{T}1\text{R}$ access transistor, ADC/DAC quantization, and read noise) and show that the cheap MVM-error feature predicts its real task accuracy on held-out configurations ($R^2 = 0.96$), turning the envelope into a predictive design rule. The individual phenomena are established in prior work; our contribution is the calibrated, baselined,

seed-averaged study and the operating envelopes that make the remedy boundaries explicit and quantitative. All experiments run in minutes on a laptop CPU and the code is publicly available on GitHub.

Index Terms—memristor, RRAM, crossbar array, in-memory computing, analog computing, device variability, IR drop, retention, in-situ training, neuromorphic.

I. INTRODUCTION

The dominant energy cost of neural-network inference on conventional hardware is not arithmetic but data movement: weights are shuttled between memory and a separate arithmetic unit across a bus, the von Neumann bottleneck. Analog resistive crossbars remove this cost by storing each weight as the conductance of a nonvolatile device at the intersection of a wordline and a bitline. When input voltages are applied to all wordlines at once, Ohm’s law turns each device into an analog multiplier and Kirchhoff’s current law sums the products along each bitline, so a full matrix-vector multiply (MVM) completes in a single read, in place, with no per-element fetch [4], [5].

Aluminium oxide (AlO_x) and related transition-metal oxides implement this conductance as a filamentary valence-change effect: an applied field drives oxygen vacancies into or out of a conductive filament bridging two electrodes, switching the cell between a low-resistance state (LRS) and a high-resistance state (HRS) [3]. The same physics that makes the device a nonvolatile memory also makes it an imperfect computing element. Three non-idealities dominate in practice: device-to-device and cycle-to-cycle variability, finite wordline and bitline wire resistance that causes an IR voltage drop across the array, and thermally activated relaxation of the stored state (retention loss). A large literature reports each effect and proposes point mitigations, and several landmark demonstrations have trained networks directly on hardware to tolerate them [9]–[11].

What is often missing, especially for a newcomer to the field, is a single, internally consistent account that runs from the device current-voltage curve up to a trained network and treats the three non-idealities on the same footing so that their distinct characters become visible. This paper provides that account. We deliberately use one device model everywhere and add each non-ideality as a small, named extension, so

that every system-level number can be traced back to a device equation. Our contributions are:

- 1) A compact AlO_x valence-change model (one state variable, threshold-gated switching, ohmic LRS blended with a nonlinear HRS) that reproduces the pinched hysteresis loop, its frequency-dependent collapse, and analog multilevel states, and that vectorizes unchanged to a 4096-cell array (Section III, IV).
- 2) A full crossbar IR-drop solver based on modified nodal analysis (MNA) of the $2NM$ -node resistive network, used to quantify how MVM error grows with array size (Section VI-D).
- 3) An Arrhenius retention model and a controlled, seed-averaged comparison that places the three non-idealities on one footing and shows how they separate by remedy under this model: variability is removed by closed-loop programming, IR drop is absorbed by hardware-in-the-loop training, and retention is only deferrable (Sections VI-E, VI-F, IX).
- 4) A reproducible demonstration, with a baseline, that on-chip training recovers a network from 63.5 to 92.4 percent under wire resistance, while a cheap output calibration recovers only to 69.3 percent (so the recovery is not a calibration artifact), and a matched demonstration that the same training is powerless against retention drift (Section VI-F).
- 5) Calibration of the model to a published self-rectifying Ag_2O crossbar [16] (reproducing its measured MNIST accuracy, cross-checked against AIHWKit), two-dimensional *operating envelopes* that map where each remedy is needed in (array size, wire resistance) and (time, temperature), and an energy model (Section VII).
- 6) An inference-only mapping of a vision-based robotics steering network with full system realism (layer tiling, a 1T1R access transistor, ADC/DAC quantization, read noise), whose real-task-accuracy operating envelope is validated out-of-sample ($R^2 = 0.96$), turning the envelope from a descriptive map into a predictive design rule (Section VIII).

We stress what is and is not new. Each phenomenon here is established: program-verify mitigates programming variability; IR drop grows with array size, conductance density and wire resistance; in-situ and hardware-aware training absorb crossbar non-idealities including wire resistance [9], [12]–[14]; and retention loss requires refresh or drift-aware compensation [15]. Our contribution is not a new mechanism but a single consistent model used end-to-end, multi-seed evidence, and an explicit calibration baseline, so that the differing characters of the three effects become visible in one place. This is a simulation study with an intentionally simplified model; the goal is reproducible clarity, not silicon-accurate prediction. Limitations are in Section X. All code and scripts to reproduce every figure and number in this paper are available at <https://github.com/rayjyotir5/alox-crossbar-nonidealities>.

II. BACKGROUND AND RELATED WORK

Memristors and RRAM. The memristor was posited by Chua as the fourth passive element linking charge and flux [1] and connected to thin-film oxide devices by Strukov et al. [2]. Practical resistive RAM (RRAM) cells switch by forming and rupturing conductive filaments; Ielmini and others review the mechanisms, reliability, and scaling [3], [4]. Compact models such as the Yakopcic model [6] capture the nonlinear conduction and threshold switching that we adopt in reduced form.

Crossbar accelerators. Architectures including ISAAC [7] and PRIME [8] use RRAM crossbars as MVM engines for deep networks, and fully hardware-implemented memristor convolutional networks have been demonstrated [10], along with large-scale analog signal and image processing [11]. Recent experimental passive crossbars are directly comparable in scale to our simulation: Keshari et al. fabricate a self-rectifying Ag-oxide 64×64 array with multilevel conductance (~ 3.6 to $22.3 \mu\text{S}$) and report 96.08 percent MNIST accuracy under device constraints, using self-rectification to suppress the sneak-path and IR-drop problems we quantify [16]. These works also document the non-idealities that motivate ours.

In-situ and hardware-aware training. Prezioso et al. trained a single-layer perceptron directly on a metal-oxide memristor array [9], and Li et al. demonstrated efficient self-adaptive in-situ learning in multilayer memristor networks [12], both establishing that training in the loop tolerates device imperfection. Hardware-aware training that models non-idealities (including wire IR drop) during training and adapts the network around them is now standard: He et al. inject modeled ReRAM non-ideal effects during mapping [13], and Rasch et al. scale hardware-aware training across large DNNs and a broad set of analog in-memory non-idealities [14]. Drift-aware inference for phase-change memory is likewise established [15]. Our hardware-in-the-loop IR-drop result is firmly in this tradition; we do not claim the phenomenon, but we add a controlled calibration baseline and seed statistics within a single device-to-system model, and we contrast it explicitly with retention, which the same loop cannot fix.

III. FROM A MEMRISTOR TO A LEARNING ARRAY

A. How a memristor works

A resistor obeys $I = V/R$ with R fixed, so its current-voltage curve is a single straight line with no memory. Chua’s argument for a fourth fundamental element [1] is that a device can instead link flux and charge, making its resistance depend on the history of charge that has passed through it: the memristor remembers. In an oxide cell this memory is physical. The film between the two electrodes is normally insulating, but a sufficient field drives charged oxygen vacancies into a conductive filament that bridges the electrodes; a reverse field ruptures it. A complete filament is the low-resistance state (LRS), a ruptured one the high-resistance state (HRS), and partial filaments give a continuum of analog states in between.

Three consequences follow, and they are the spine of this paper. First, because moving ions takes a threshold field and

finite time, a slow voltage sweep traces a *pinched hysteresis loop*: the resistance on the way up differs from the way down, yet both branches pass through the origin because no current flows at zero voltage (Section VI-A). Second, when the voltage returns to zero the filament stays put, held by an energy barrier, so the conductance is a *nonvolatile stored value*; raising temperature lowers that barrier and the state slowly relaxes, which is retention loss (Section VI-E). Third, that stored conductance is exactly what a neural network needs as a *weight*, and an array of such cells multiplies an input vector by a weight matrix in one electrical step (Section IV). The rest of the paper builds on these three facts in order: the device, then the array and its non-idealities, then inference, then on-chip learning.

B. Compact device model

We model the cell with a single internal state variable $w \in [0, 1]$ that represents filament completeness: $w \rightarrow 1$ is the LRS (filament bridges the oxide) and $w \rightarrow 0$ is the HRS (filament ruptured). The terminal current blends an ohmic LRS branch with a sinh-nonlinear HRS branch, the behaviour AlO_x cells show on a parameter analyzer:

$$I(V, w) = w \frac{V}{R_{\text{on}}} + (1 - w) \frac{V_0}{R_{\text{off}}} \sinh\left(\frac{V}{V_0}\right), \quad (1)$$

where R_{on} and R_{off} are the LRS and HRS resistances and V_0 shapes the HRS nonlinearity. The state evolves only when the bias exceeds a SET or RESET threshold, which is the origin of both hysteresis and nonvolatility:

$$\frac{dw}{dt} = \begin{cases} k_s(V - V_{\text{set}})(1 - w^p), & V > V_{\text{set}} \\ -k_r(-V - V_{\text{reset}})(1 - (1 - w)^p), & V < -V_{\text{reset}} \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

The window factors $(1 - w^p)$ and $(1 - (1 - w)^p)$ ease the dynamics near the rails and produce the gradual, saturating, asymmetric conductance change between potentiation (SET) and depression (RESET) that is characteristic of real cells and central to in-situ learning. A small-signal read at a sub-threshold voltage V_{read} leaves w unchanged, so reading is nondestructive. Default parameters are listed in Appendix A.

IV. CROSSBAR ARRAY AND NON-IDEALITY MODELS

Variability. An $N \times N$ array instantiates Eq. 1 and 2 per cell with parameters drawn from realistic distributions: log-normal R_{on} and R_{off} (15 and 30 percent spread, the HRS being the more variable), and Gaussian SET and RESET thresholds. A multiplicative cycle-to-cycle term perturbs each programming pulse.

Differential weights. Signed weights are mapped to a pair of cells, $W \propto (G^+ - G^-)$, so that one bitline pair encodes positive and negative contributions; this also cancels common-mode disturbances.

Closed-loop programming. To write a target conductance map we use program-verify: read every cell, then apply a SET pulse to those below target and a RESET pulse to those above,

repeated until convergence. Residual error reflects finite pulse granularity and write noise rather than raw device spread.

Wire IR drop. We solve the full resistive network rather than assuming ideal Ohm-Kirchhoff behaviour. Each cross-point contributes a row-wire node and a column-wire node; adjacent nodes along a wire are joined by a segment conductance $g_w = 1/r_w$, and the device G_{ij} links the two node planes. Driving rows from their left ends and grounding columns at their feet yields a $2NM$ -unknown linear system assembled by modified nodal analysis. Because the conductance matrix depends only on G and r_w , we factor it once (sparse LU) and reuse the factorization across many input vectors, which makes hardware-in-the-loop training tractable.

Retention. The stored state relaxes toward equilibrium with an Arrhenius time constant,

$$w(t) = w_{\text{eq}} + (w_0 - w_{\text{eq}}) e^{-t/\tau(T)}, \quad \tau(T) = \tau_0 e^{E_a/k_B T}, \quad (3)$$

so a programmed LRS cell loses conductance over time, faster when hot. We use $E_a = 1.0$ eV.

V. EXPERIMENTAL SETUP

All experiments use NumPy, SciPy (sparse LU for the IR-drop solve), scikit-learn (the digit dataset and the reference classifier), and PyTorch (a reference convolution), on a laptop CPU; the full pipeline runs in minutes. Code is organised as a device-and-array module, a non-idealities module (IR drop, retention, inference), and an in-situ learning module; see Appendix D.

VI. RESULTS

A. Single-Device Behaviour

Figure 1 shows a quasi-static triangular sweep. Panel (a) is the pinched hysteresis loop, the defining memristor signature: both branches cross the origin because $I = 0$ whenever $V = 0$. Panel (b) shows the state variable switching abruptly at the SET and RESET thresholds, and holding flat elsewhere, which is why a single voltage maps to two resistances (hysteresis) and why the state survives at zero bias (memory). Panel (c) shows the loop collapsing as sweep frequency rises: at 1, 10, and 100 Hz the peak state reaches 1.0, 0.65, and 0.09 respectively, because the ions cannot keep up and the device stops switching.

B. Array Variability

Driving all 4096 cells fully off and then fully on yields the conductance histograms of Fig. 2(a). The HRS reads $7.09 \mu\text{S}$ ($\sigma/\mu = 0.24$) and the LRS $199 \mu\text{S}$ ($\sigma/\mu = 0.19$); although the means differ by about 28x, the worst-case on/off read margin (lowest LRS over highest HRS) is only about 4x. This margin erosion, not the mean ratio, is the practical variability problem. Panel (b) overlays the loops of 60 random cells: no two are identical.

AlO_x single device — Pt / Al₂O₃ / Ti

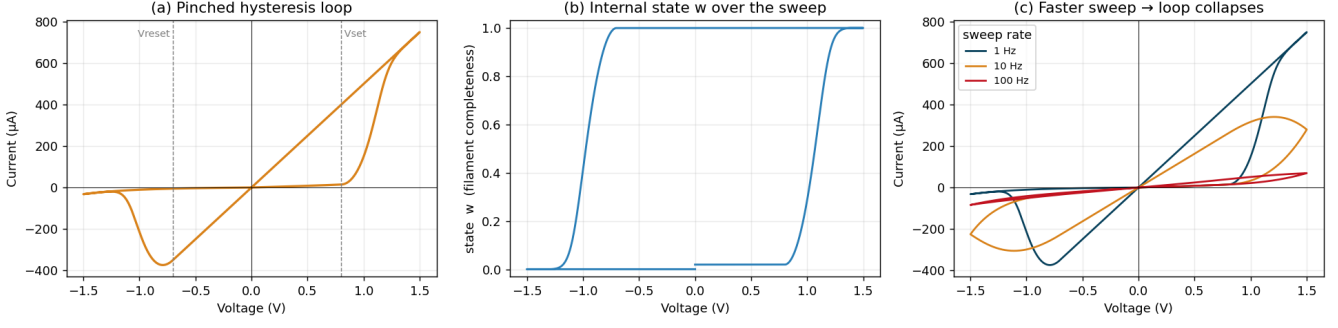


Fig. 1. Single AlO_x cell. (a) Pinched hysteresis loop. (b) State variable w over the sweep, showing threshold-gated SET and RESET. (c) The loop area collapses monotonically as sweep frequency increases and the state can no longer fully switch.

64×64 array (4096 cells) — device-to-device variability

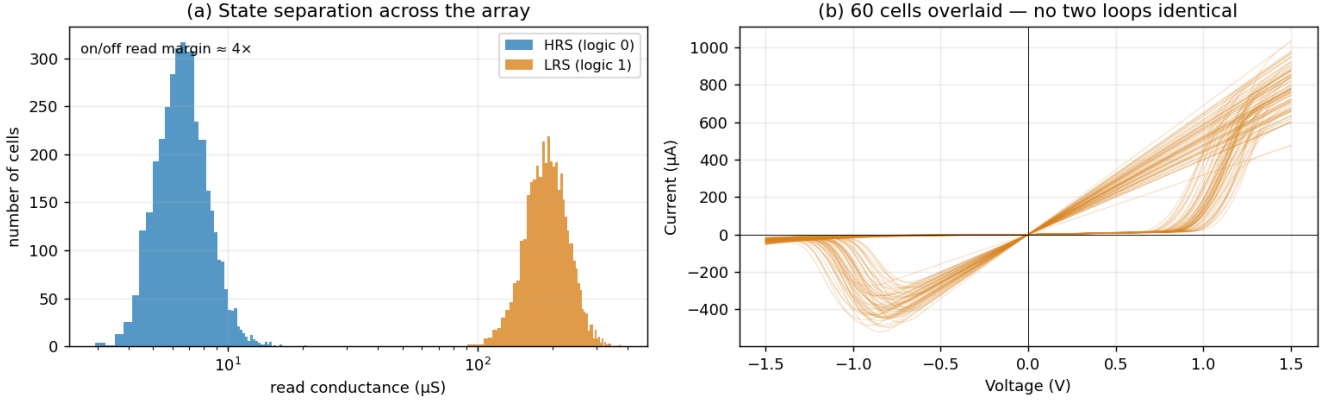


Fig. 2. 64×64 array (4096 cells). (a) HRS and LRS read-conductance distributions; the worst-case on/off margin (about 4x) is far smaller than the ratio of means (about 28x). (b) Current-voltage loops of 60 randomly chosen cells, showing device-to-device spread.

C. In-Memory MVM and Classifier Inference

Figure 16 programs an image into the array as a conductance map and reads it back; the program-verify write achieves about 0.5 percent RMSE of full scale. The analog MVM (input voltages times the programmed conductances, summed by column) tracks an ideal digital MVM to about 0.5 percent mean error.

We then deploy a real classifier whose single layer fits the array. A logistic regression on 8×8 handwritten digits (64 inputs, 10 classes) is trained in software (96.5 percent test accuracy), mapped to differential conductances, and evaluated on the array. Table I reports the accuracy under each condition. An ideal write reproduces the software accuracy, confirming the pipeline. Open-loop writing (which lets device variation pass through uncorrected) costs about 8 points (88.3 ± 2.6 over five seeds), and program-verify recovers essentially all of it (96.5 ± 0.3). Adding wire IR drop is then the larger loss. As a building-block check, a single convolutional layer (the dominant operator in detectors such as YOLO) mapped to the array reproduces a floating-point convolution with cosine

TABLE I
DIGIT CLASSIFIER (64 TO 10) UNDER ARRAY NON-IDEALITIES (5 SEEDS, MEAN $\pm$ STD).

Condition	Test accuracy (%)
Software (float)	96.5
Crossbar, ideal write	96.5
Open-loop write (raw variability)	88.3 ± 2.6
Program-verify write	96.5 ± 0.3
+ wire IR drop ($r_w = 2.5 \Omega$)	75.1 ± 1.0

similarity 0.999, but a full detector neither fits in nor can be loaded onto a single small tile, so we report the layer fidelity and the end-to-end classifier rather than a detector.

D. Wire IR-Drop Scaling

Figure 5(a) maps the wordline voltage across a 64×64 all-LRS array driven at 200 mV: the voltage sags from the driven edge to about 50 mV at the far corner, so distant cells are under-driven and contribute too little current. Panel (b) and Table II show that the resulting MVM error grows sharply

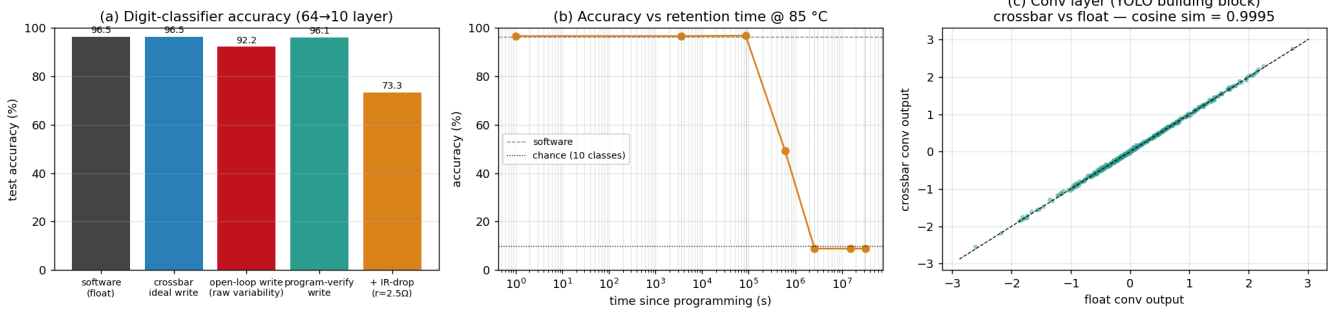


Fig. 3. Inference on the array. (a) Classifier accuracy across conditions: variability costs accuracy, program-verify recovers it, and IR drop then dominates. (b) Accuracy versus retention time at 85 degrees Celsius. (c) A convolutional layer (a YOLO building block) on the array reproduces the floating-point output with cosine similarity 0.999.

Test-time inference on the crossbar: the spiral is segmented point by point (left to right)

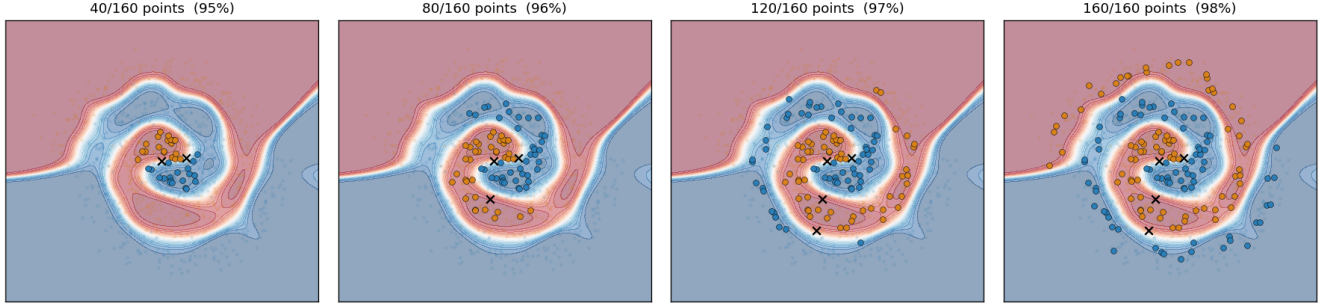


Fig. 4. Test-time inference unfolding in time (frames left to right). A held-out spiral is pushed through the trained crossbar point by point (revealed from the centre outward); each point is classified and placed on the plane, coloured by the array's prediction (black cross if wrong), over the learned decision boundary. By the last frame the two arms are cleanly separated.

TABLE II
MEAN MVM ERROR (%) VS. ARRAY SIZE (5 SEEDS; STD < 0.3).
ALL-LRS IS THE WORST CASE; MIXED-STATE IS A REALISTIC RANDOM CONDUCTANCE MAP.

Pattern, r_w	16x16	32x32	64x64	128x128
all-LRS, 1.0 Ω	8.6	26.1	56.6	82.4
all-LRS, 2.5 Ω	18.8	46.1	75.5	91.5
all-LRS, 5.0 Ω	31.5	62.5	85.3	95.2
mixed-state, 2.5 Ω	6.9	22.1	51.1	79.0

with array size, because both the wire length and the aggregate current increase. This is the worst case (every cell in LRS); a realistic mixed-state map roughly halves the error (for example 51.1 versus 75.5 percent at 64x64, Table II), but the monotone growth with size holds for both and explains why passive arrays are size limited and why access transistors, current limiting, or self-rectifying cells [16] are used in practice.

E. Retention

Figure 6(a) shows normalized LRS conductance versus time at four temperatures. The Arrhenius time constant is about 12 years at 25 degrees Celsius, 6.5 days at 85 degrees, and 6

hours at 125 degrees. Panel (b) illustrates a stored weight pattern fading to nothing over a month at 85 degrees: the device forgetting in real time. The absolute time scale depends on the activation energy: holding 25-degree retention near ten years, the 85-degree time constant is 19.8, 5.4, and 1.5 days for $E_a = 0.8, 1.0$, and 1.2 eV respectively, so the qualitative cliff is general while its location is set by E_a and temperature.

F. In-Situ Learning and the Character of Each Non-Ideality

We now ask how the non-idealities interact with training, using a two-spiral classification task that requires a genuinely nonlinear network (a linear model scores about 50 percent). A 2-64-2 multilayer perceptron is trained with its weights stored on differential crossbar tiles; every weight update is a programming pulse that moves w through the same non-linear dynamics of Eq. 2, so the update inherits the device's saturating, asymmetric, variable response. We use a gradient-proportional pulse rule (pulse width scales with the gradient magnitude).

Training converges on imperfect devices. The variable, noisy crossbar network learns the task, but reliability depends on the pulse schedule. With a fixed pulse size the spiral accuracy is seed-sensitive (72.4 ± 13.1 over four seeds);

A. Wire IR-drop — full nodal solve of the crossbar network

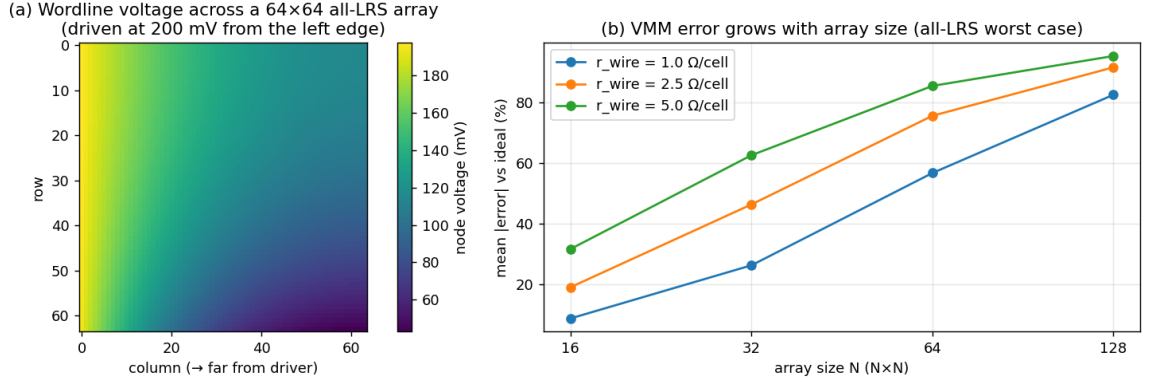


Fig. 5. Wire IR drop from the full nodal solve. (a) Wordline voltage across a 64×64 all-LRS array driven at 200 mV; the voltage droops toward the far corner. (b) Mean MVM error versus array size for three wire resistances (worst case).

B. Retention / drift — the state relaxes thermally over time

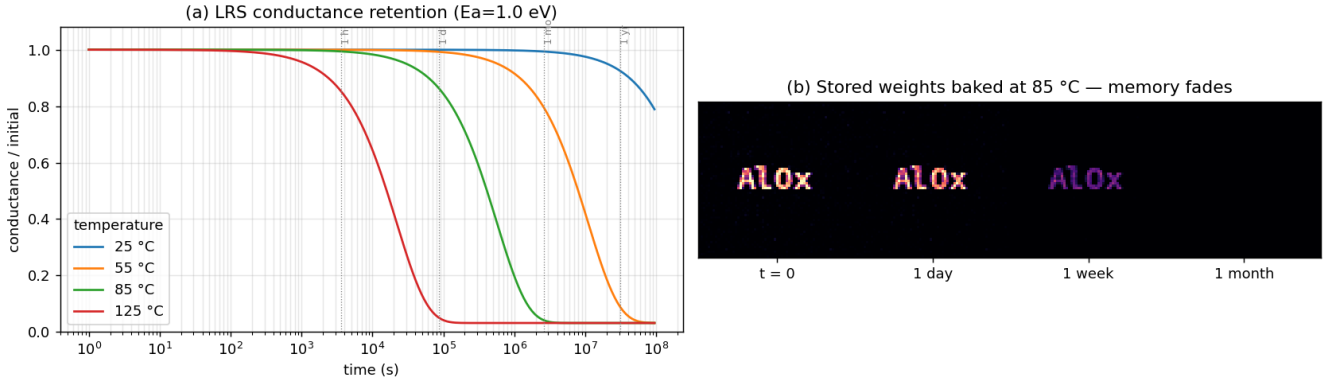


Fig. 6. Retention. (a) Normalized LRS conductance versus time at four temperatures; each temperature shifts the decay by roughly a decade. (b) A stored weight pattern baked at 85 degrees Celsius fades from sharp to gone over a month.

adding a pulse-size decay schedule stabilises it to 95.2 ± 1.2 , against a floating-point baseline near 95 percent. On an easier second nonlinear task (concentric circles) training is robust either way (99.9 ± 0.1). Figure 7 shows a representative seed. Two mechanisms matter. First, the usable weight range: the conductance window must encode weights up to about ± 8 for the spiral task, and clipping it to ± 3 caps accuracy near 80 percent. Second, device noise helps: noiseless, identical devices stall in limit cycles under deterministic nonlinear updates, whereas cycle-to-cycle noise acts as beneficial dither, a known and somewhat counterintuitive in-situ result.

IR drop is absorbable by training, and not by calibration alone. A network trained with an ideal forward pass and then deployed through the IR-drop network sits at 63.5 ± 0.9 percent at $r_w = 1 \Omega$ (three seeds). Training the same network with the IR-drop network in the forward pass (gradients from an ideal-weight surrogate, since one cannot backpropagate through the analog array) recovers it to 92.4 ± 0.9 percent (Fig. 8(b),(c)). To

check that this is genuine compensation and not a trivial output rescaling, we add a baseline: fit a per-output affine map (gain and bias) from the IR-distorted logits to the ideal logits on half the data and evaluate on the rest. This output calibration recovers only to 69.3 ± 1.5 percent, far below the trained 92.4. So the recovery requires learning weights tuned to the distorted transfer function, not a cheap post-hoc correction. A pulse-size decay schedule is needed to settle the otherwise noisy convergence caused by the surrogate-gradient mismatch.

Retention is not. Taking the trained network and letting its conductances relax (Fig. 8(a)), the accuracy holds for about a day at 85 degrees, falls through 60 percent at one week, and reaches chance (50 percent) by one month. A static set of weights cannot resist this, because the distortion is time varying rather than fixed; it is deferred by periodic rewriting, or mitigated by drift-aware mapping, recalibration, or refresh policies, but not removed by the one-time hardware-aware training that absorbs IR drop.

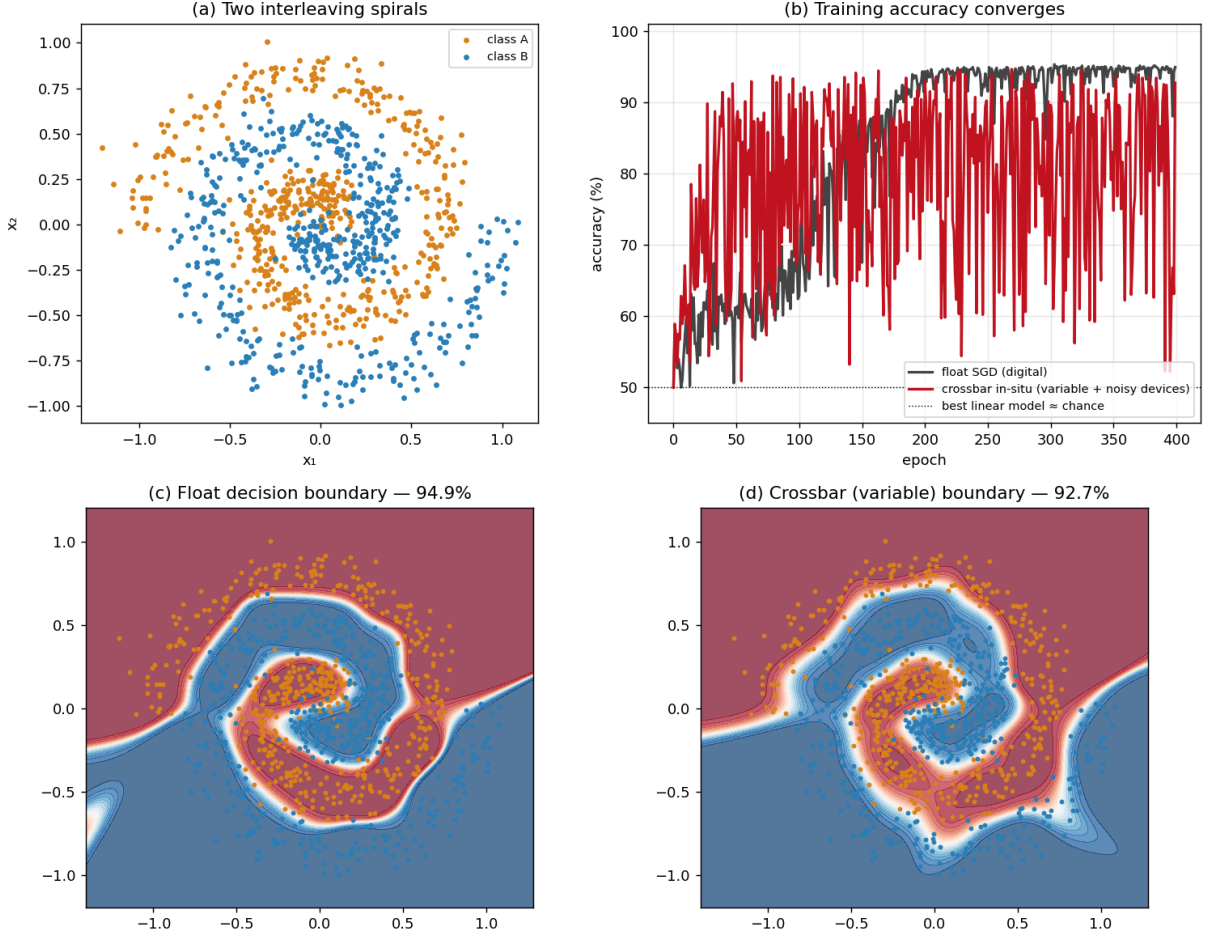


Fig. 7. In-situ learning on the crossbar (a representative seed; see text for seed statistics). (a) The two-spiral task. (b) Training accuracy converges for both floating-point SGD and the variable, noisy crossbar. (c, d) Learned decision boundaries; the crossbar closely matches floating point.

VII. CALIBRATION, OPERATING ENVELOPE, AND ENERGY

The experiments so far use the generic device model. To ground the quantitative conclusions in a real device and against an accepted baseline, we now calibrate the model to a published cell, validate it, chart operating envelopes, and add an energy model.

Calibration to the published measured curves. Rather than assume device parameters, we fit them to the figures in the supplementary material of Keshari et al. [16]. From their multilevel data we take the conductance window of 3.6 to 22.3 μS and the 100 distinct conductance states they report (about 6.6 bits per cell, 9855 signed values with a differential pair). From the LRS retention failure plot (their Fig. S5b) we digitize the Arrhenius points (roughly 4×10^3 s at 373 K and their stated ten-year retention at 298 K) and fit an activation energy $E_a = 1.45$ eV; this reproduces their ten-year room-temperature retention and implies retention of only hours at 85 degrees, markedly shorter than the illustrative $E_a = 1.0$

eV used in Section VI-E. Their measured potentiation curves (Fig. S8) are nearly linear (nonlinearity $\alpha_p \approx 0.02$), consistent with the gradient-proportional update we use, and their cell distributions (Fig. S4) show the low variation we model. We map an MLP onto this device, following their setup (8x8-resized MNIST, 64 inputs), with 100-level quantization and programming noise. Table III compares accuracy: the device penalty in our simulator is 0.8 points (software 94.3 to 93.5 ± 0.3), close to the 0.6-point penalty of an established hardware-aware toolkit (AIHWKit [14], 93.7 ± 0.1) and to the near-zero penalty Keshari et al. report (96.1 to 96.08). Our absolute software baseline is below theirs, a training and architecture difference, but the quantity the calibration validates, the small loss the device itself adds, agrees across all three.

Operating envelopes. Figure 10 charts, for the calibrated device, where each remedy is needed. Panel (a) is the IR-drop envelope over array size and wire resistance: below the 2 percent MVM-error contour naive deployment suffices, between

Retention drift after training & learning around IR-drop

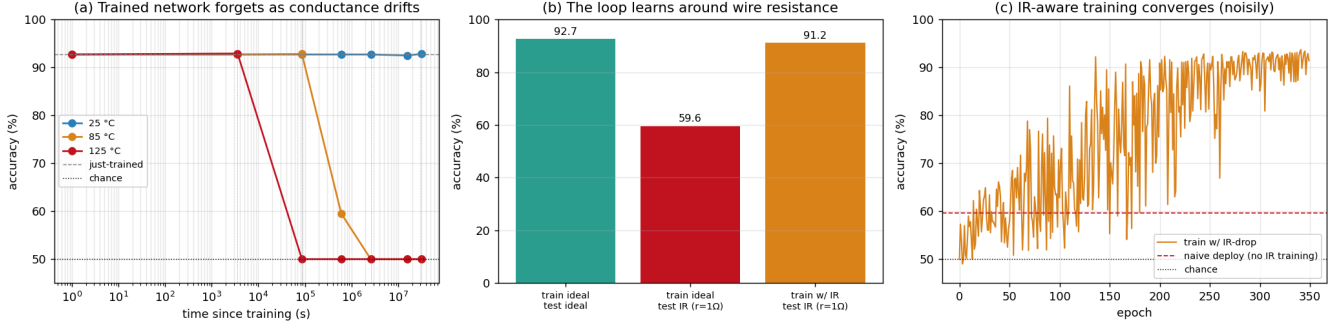


Fig. 8. Two interactions of non-idealities with a trained network. (a) Retention drift after training: accuracy holds for about a day at 85 degrees Celsius, then dissolves to chance by one month. (b) IR drop: naive deployment under wire resistance collapses accuracy, but training with IR drop in the loop recovers it. (c) The IR-aware training converges (with a pulse-size decay schedule).

Retention drift at 85 C: the trained decision boundary dissolves to chance over a month

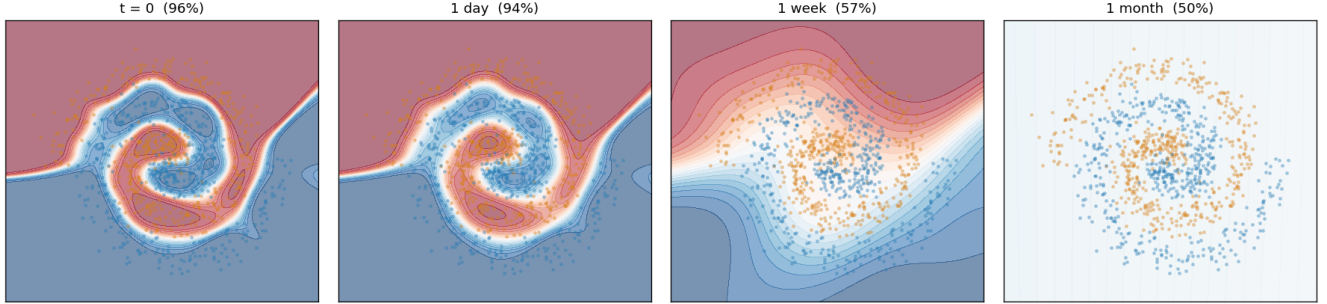


Fig. 9. The same trained network forgetting in time (frames left to right). As the conductances relax at 85 degrees Celsius, the decision boundary is intact at $t = 0$ and after one day, washes out by one week (57 percent), and is gone by one month (chance). No fixed set of weights can resist a time-varying decay.

TABLE III
8x8 MNIST MLP ON THE CALIBRATED AG₂O DEVICE; THE VALIDATED QUANTITY IS THE DEVICE PENALTY (SOFTWARE MINUS DEVICE).

Method	Accuracy (%)	Device penalty
Software (float, ours)	94.30	—
AIHWKit analog inference [14]	93.7 ± 0.1	0.6
Our calibrated simulator	93.5 ± 0.3	0.8
Keshari et al. 2026, measured [16]	96.08	0.02

roughly 2 and 30 percent the loss is recoverable by program-verify and in-situ training (Sections VI-C, VI-F), and beyond it the array is IR-limited. Panel (b) is the retention envelope over time and temperature, evaluated on the trained MLP: the 90 percent accuracy contour gives the rewrite cadence, which falls from years at room temperature to hours at 85 degrees (steep because the measured $E_a = 1.45$ eV makes retention strongly temperature-dependent). Together the two envelopes are a predictive map: given a target accuracy, they bound the usable array size, wire resistance, and rewrite interval.

One nuance worth stating: self-rectification, the headline feature of the calibrated device, does *not* reduce the read-time IR-drop error. In a fully parallel single-ended MVM

every cell is forward-biased, so the rectifying element never engages; we verified that passive and self-rectifying cells give identical MVM error (difference below 0.01 percent). Self-rectification suppresses sneak-path and write-disturb currents and so enables denser passive arrays (the benefit Keshari et al. emphasize), but the read-time IR drop is a wire-resistance phenomenon it does not address. This separates two effects that are often conflated.

Energy. Figure 11 estimates compute efficiency. With the calibrated conductances, a read pulse, and a per-column ADC, the array delivers 18 to 41 TOPS/W, rising with size as the ADC cost amortizes, against about 4 TOPS/W for a 0.5 pJ digital MAC. Energy therefore favors larger arrays, but the IR-drop energy lost in the wires climbs from under 1 percent to 21 percent across the same range, and IR drop also corrupts accuracy. The practical conclusion is that the accuracy envelope of Fig. 10(a), not energy, sets the usable array size; within that envelope the calibrated device offers roughly five to ten times the efficiency of the digital baseline.

VIII. INFERENCE-ONLY MAPPING OF A ROBOTICS NETWORK

The preceding envelopes use small classifiers. To test whether the same calibrated device model constrains a real-

Operating envelopes for the calibrated device: where each remedy is needed (IR contours 2/10/30%; retention contours 90/93%)

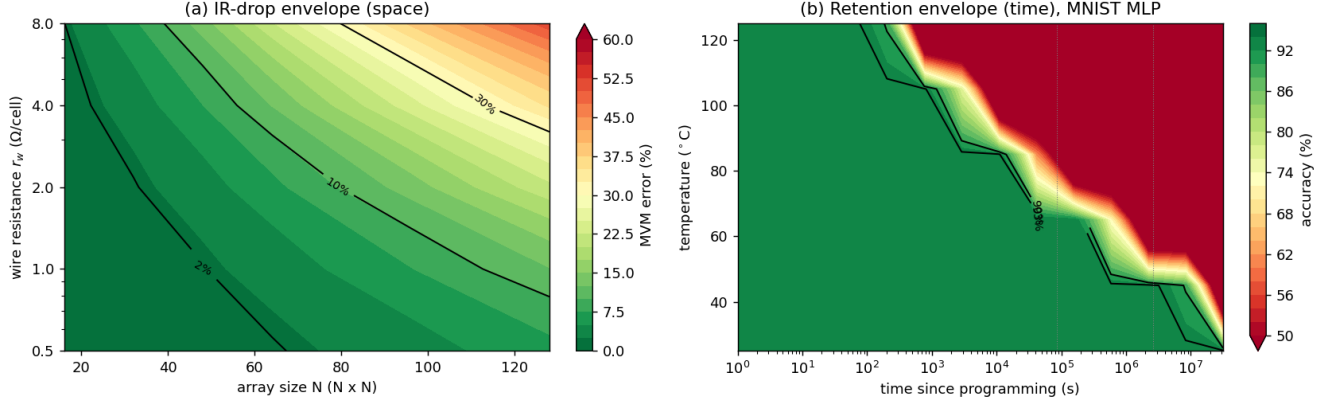


Fig. 10. Operating envelopes for the calibrated Ag_2O device. (a) IR-drop envelope: mean MVM error over array size and wire resistance, with 2/10/30 percent contours marking the naive-OK, training-recoverable, and IR-limited regions. (b) Retention envelope: accuracy of the trained MNIST MLP over time and temperature, with 90/93 percent contours giving the rewrite cadence.

Energy model: in-array efficiency and the IR-drop penalty

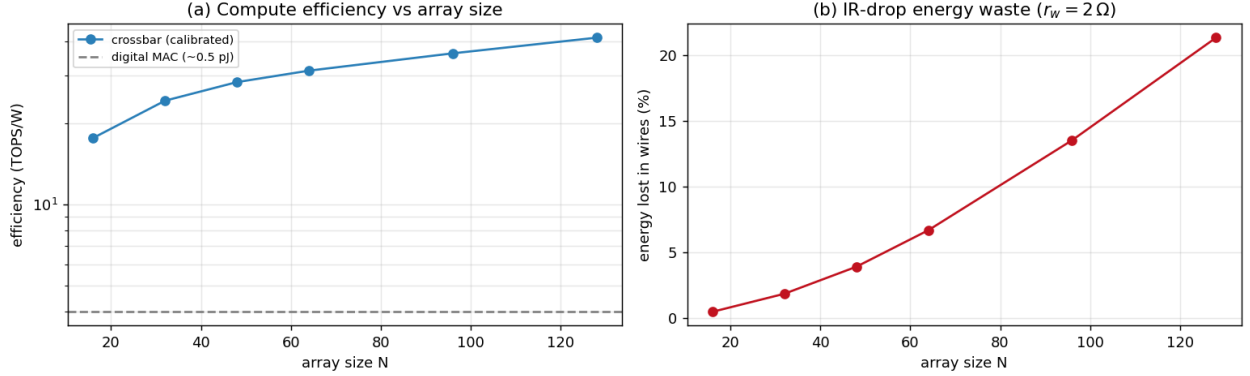


Fig. 11. Energy model for the calibrated device. (a) Compute efficiency (TOPS/W) versus array size, against a digital MAC reference. (b) Fraction of energy lost in the wires to IR drop, which grows with array size.

istic workload, we mapped a PilotNet steering convolutional network onto the Ag_2O crossbar in an inference-only setting. PilotNet is a regression network that predicts a steering angle from a forward camera frame; we use real task accuracy, defined as the fraction of held-out frames whose predicted angle falls within a fixed tolerance of the reference, as the figure of merit throughout. In software (float) the network reaches 99.9 percent on this metric, which sets the ceiling for every analog mapping below. Figure 12 demonstrates the mapped inference directly: at a good operating point the crossbar reproduces the commanded steering and stays inside the lane-keeping tolerance across a drive, while at a degraded operating point the same network drifts out of lane.

A. Mapping with system realism

Each convolutional and fully connected layer is lowered to a matrix product by `im2col` and tiled across one or more physical

arrays of size N . Every tile is evaluated through the full pipeline rather than an idealized matrix multiply: a DAC drives the rows, the column currents pass through ADC quantization, and peripheral read noise is added at the converters. The conductances are the calibrated Ag_2O values, and the column currents are obtained from the same nodal IR-drop solve used in the rest of this work, parameterized by the wire-resistance ratio r_w . We evaluate two cell options: a passive one-resistor (1R) cell, and a one-transistor one-resistor (1T1R) cell with an access transistor. We sweep 40 configurations in total: a 32-point grid over array size and wire resistance, 4 ADC/DAC bit-depth settings, and 4 ideal references.

B. Operating envelope on real task accuracy

Figure 13 shows the envelope. Real task accuracy collapses monotonically as array size N and wire resistance r_w grow: for the passive 1R cell at $r_w = 2$ it falls from 99.3 percent at

Steering inference on the crossbar: road frame -> commanded steering

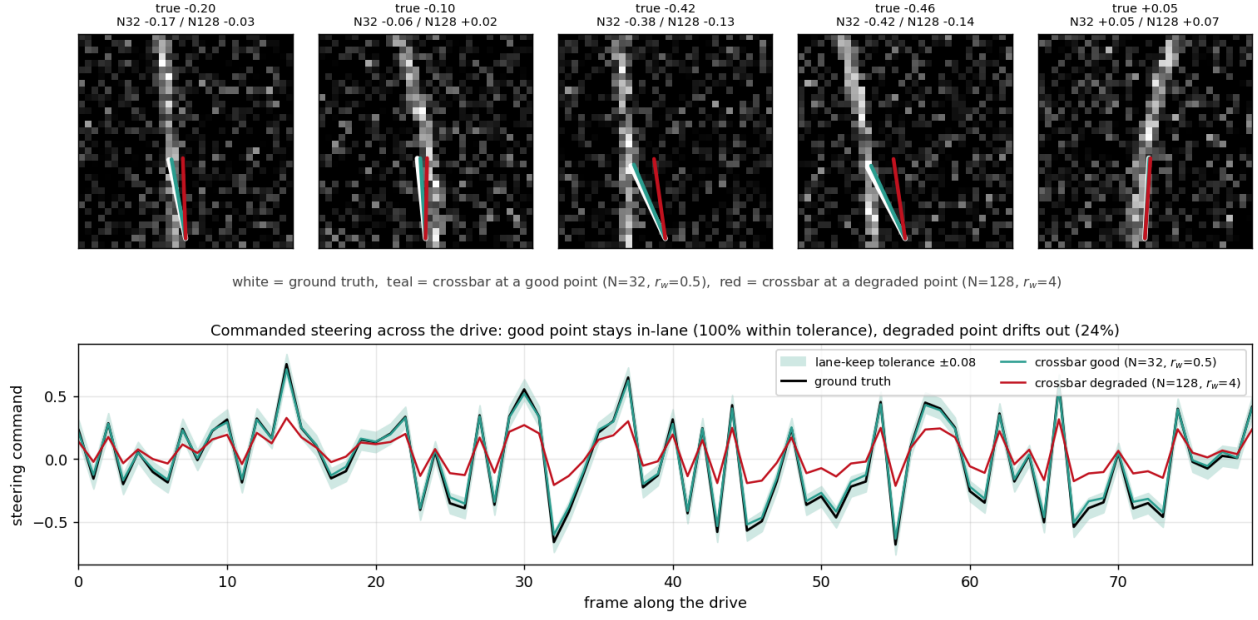


Fig. 12. The steering network running inference on the crossbar. Top: five road frames, each annotated with the ground-truth steering (white) and the crossbar-commanded steering at a good operating point (teal, $N = 32, r_w = 0.5$) and a degraded one (red, $N = 128, r_w = 4$). Bottom: the commanded steering across a drive. At the good point the crossbar tracks the reference inside the lane-keeping tolerance band (100 percent of frames in tolerance); at the degraded point it drifts out (24 percent), which would steer the vehicle out of its lane.

$N = 32$ to 66.7, 49.3, and 34.7 percent at $N = 64, 96$, and 128. Steering regression tolerates far less analog error than the digit and spiral classifiers studied earlier, so the largest 1R tile within two points of the float ceiling at $r_w = 2$ is only $N = 32$; a demanding regression workload narrows the usable envelope sharply. The 1T1R cell is consistently more robust at large arrays (72.0, 56.0, 37.3 percent at $N = 64, 96, 128$, versus 66.7, 49.3, 34.7 for 1R), because the access transistor limits per-cell current and so reduces the IR drop. The gain is modest because the same series resistance compresses the device’s already narrow conductance window; this is the expected access-device trade-off, made quantitative.

C. ADC/DAC ablation

Figure 15 isolates converter precision. Holding $N = 96$, $r_w = 2$, and the 1R cell fixed, real task accuracy is essentially flat (about 48 to 55 percent) from 4-bit to full precision. In this IR-dominated regime the analog conductance and wire-resistance non-idealities, not converter bit depth, set accuracy. Spending bits on higher-resolution converters does not move the operating point; relieving IR drop (smaller tiles, or the 1T1R cell) does.

D. Out-of-sample validation

An envelope is only useful if it predicts configurations not used to build it. We test whether the cheap, network-independent MVM-error feature, a single small random-tile solve that needs no network run, predicts the real task accuracy of the full mapped network. Fitting a degree-2 curve of

accuracy against $\log_{10}(\text{MVM error})$ on a random 60 percent of configurations and predicting the held-out 40 percent gives a held-out $R^2 = 0.96$ and a mean absolute error of 3.9 accuracy points (Pearson -0.92). The prediction holds both within the 1R family and pooled across 1R and 1T1R, with the 1T1R points sitting slightly above the curve by the small IR-relief margin (Fig. 14). The feature therefore turns the envelope from a descriptive map into a predictive design rule: a new operating point can be placed on the envelope, within a few accuracy points, without running the network. That is what makes the envelope a tool rather than a chart, and it is the central result of this section.

IX. DISCUSSION: SEPARATION BY REMEDY

Within this model and these tasks, the three non-idealities tend to separate by the kind of remedy that helps, a separation made visible by holding the device model fixed across all three:

- **Variability** behaves as a largely static, per-device offset, and is *correctable at write time* by closed-loop programming (program-verify restores the classifier from 88.3 to 96.5 percent). This holds for the programmable variation modeled here; stuck or non-responsive devices, read noise, and limited verify budgets would leave a residual.
- **IR drop** is a static, structured, input-dependent distortion of the transfer function. It is not a write problem and program-verify does not fix it, but it is largely *absorbable*

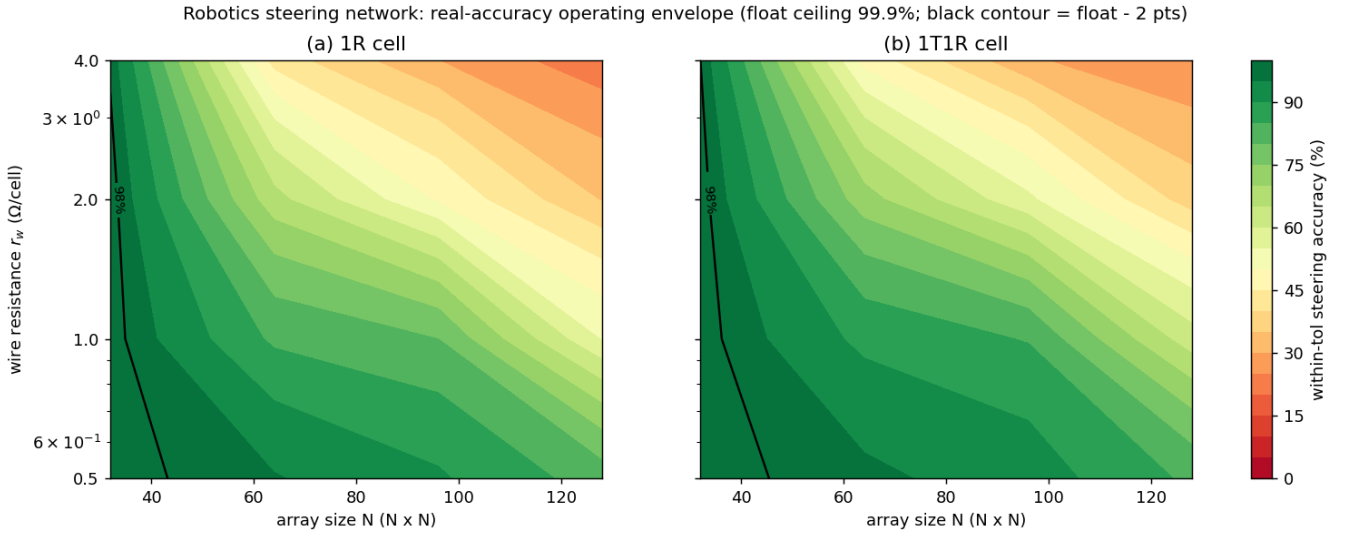


Fig. 13. Real task accuracy envelope for the PilotNet steering network mapped inference-only onto the calibrated Ag_2O crossbar. Accuracy is shown over array size N and wire resistance r_w for the passive 1R cell and the 1T1R cell. Both collapse monotonically with N and r_w ; the 1T1R cell is consistently more robust at large arrays because its access transistor relieves the IR drop. The black contour marks the float ceiling minus two points.

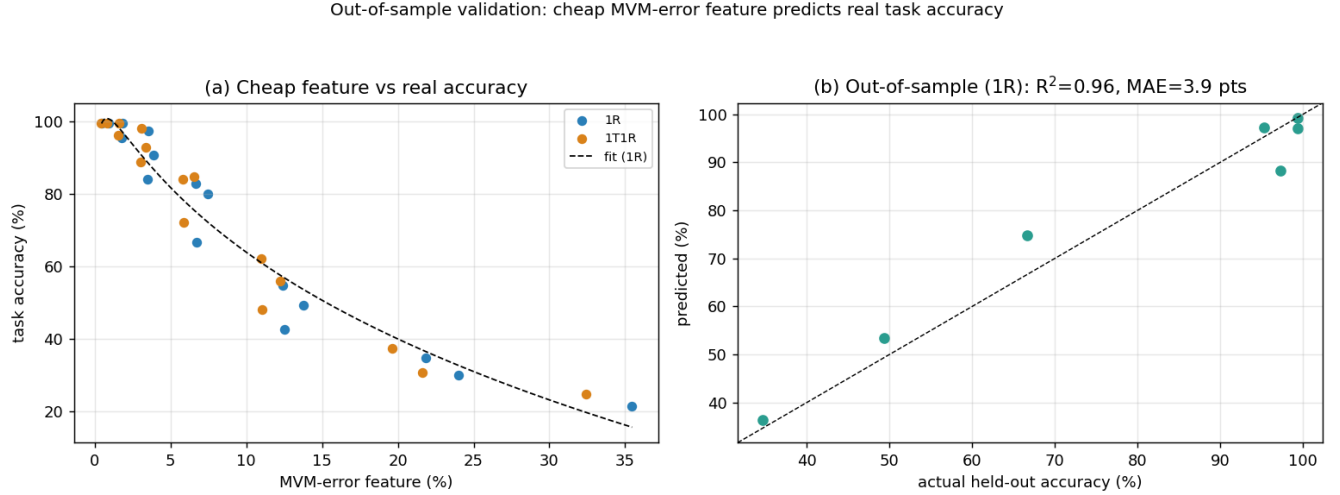


Fig. 14. Out-of-sample validation of the cheap MVM-error feature against real task accuracy. (a) Real accuracy versus the feature for all grid configurations, with the degree-2 fit. (b) Held-out predictions versus actual accuracy lie on the diagonal: $R^2 = 0.96$, MAE 3.9 points, so the feature predicts the real task accuracy of configurations it never saw, making the envelope predictive.

by training (63.5 to 92.4 percent, versus 69.3 for output calibration alone), provided the array is not so resistive that columns saturate. It still sets a practical ceiling on passive array size (Table II).

- **Retention** is a time-varying decay, orthogonal to both: a fixed set of trained weights cannot resist it, so it is *deferred* by rewriting or addressed by drift-aware mapping and recalibration, on a cadence set by the activation energy and temperature.

The practical reading is that the three call for different mechanisms, and that expecting one remedy to cover another (better programming to fix IR drop, or one-time hardware-aware training to fix retention) is unlikely to work. We state

this as an observation under our model rather than a universal law.

X. LIMITATIONS

This is a simulation study with a reduced model and several intentional simplifications. The device model uses a single state variable and a simple threshold rule; it captures hysteresis, multilevel states, frequency dependence, and asymmetric updates, but not the full stochastic filament dynamics, read disturb accumulation, or $1/f$ noise. The IR-drop study assumes a passive 1R array and an all-LRS worst case for the scaling table; a 1T1R cell with an access transistor would relax these numbers. Retention uses a single activation energy and a

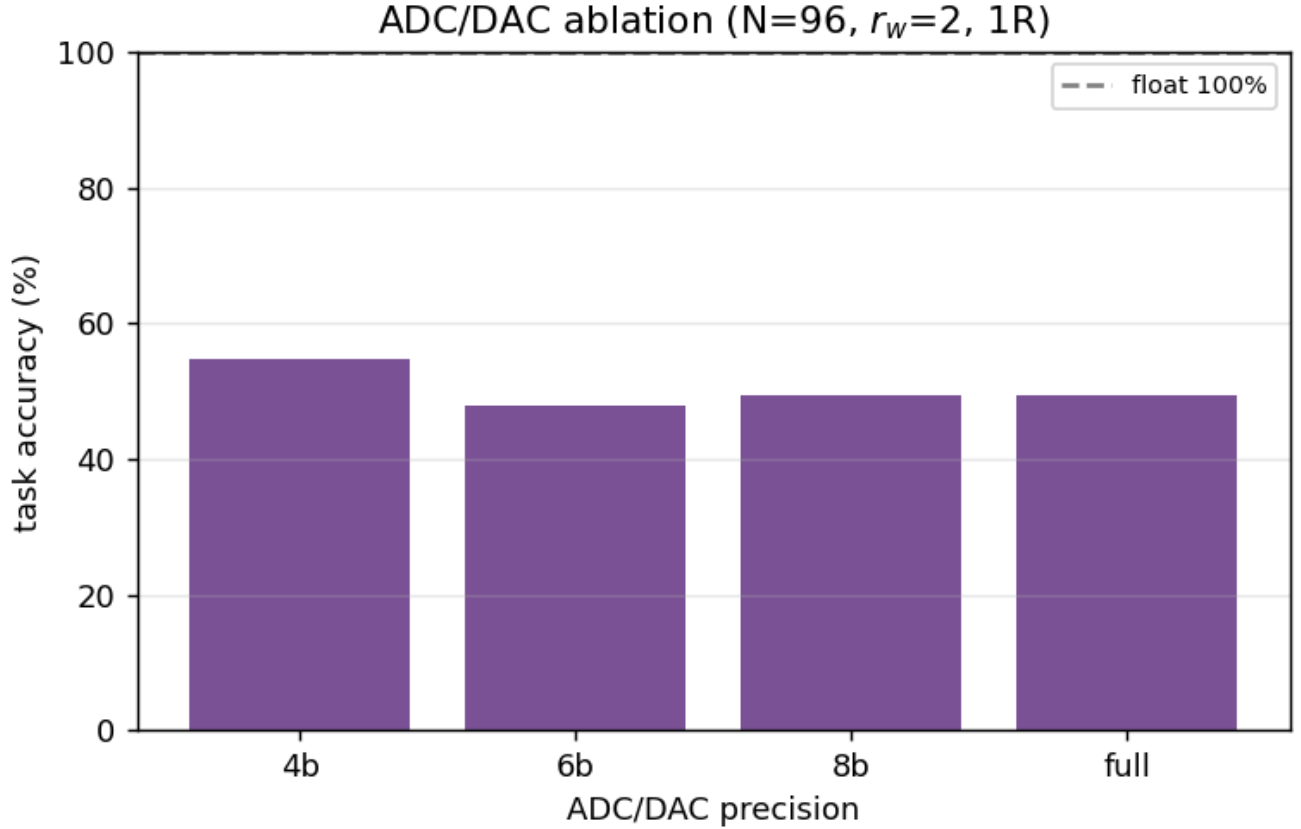


Fig. 15. ADC/DAC ablation at $N = 96$, $r_w = 2$, 1R cell. Real task accuracy is flat near 48 to 55 percent across 4, 6, 8, and full-precision converters, indicating that analog conductance and wire-resistance non-idealities, not converter bit depth, set accuracy in this IR-dominated regime.

relaxation toward the HRS; real distributions are broader. The in-situ backward pass uses an ideal-weight surrogate gradient rather than a measured one; the calibration baseline (Section VI-F) shows the recovery is not a trivial rescaling, but a measured or finite-difference gradient was not compared. The tasks (digits, two spirals, circles) are small enough to fit a single tile; we report a convolutional-layer fidelity rather than a full detector. We calibrate by *digitizing the published measured figures* of the device (the Arrhenius retention plot for E_a , the multilevel and cell-distribution plots, the potentiation curves); the authors release no raw per-point data files, so the fit carries figure-reading error, and the energy model is a first-order estimate (read power plus a per-column ADC) without full peripheral, 1T1R, or interconnect parasitics. None of the three underlying phenomena is itself novel; each is in the literature of Section II. The contribution is the calibrated, AIHWKit-baselined, seed-averaged study and the operating envelopes that make the remedy boundaries quantitative. Remaining steps toward a top-venue result are fitting the model to raw measured data, a full peripheral and 1T1R model with silicon-grade energy, and larger networks.

XI. CONCLUSION

Starting from one physically grounded oxide-memristor device equation, calibrated to a published Ag_2O cell, and scaling it without modification to a 64×64 array and a trained network, we gave a unified, seed-averaged account of three crossbar non-idealities. Under this model they tend to separate by remedy: variability is corrected at write time (88.3 to 96.5 percent), IR drop is absorbed by hardware-in-the-loop training (63.5 to 92.4 percent, where output calibration alone reaches only 69.3), and retention is only deferrable. The calibrated simulator reproduces the published 96.08 percent MNIST result to within about a point and agrees with AIHWKit, and the resulting operating envelopes map where each remedy is needed over array size, wire resistance, and temperature. None of the underlying phenomena is individually new; the value is the calibrated, baselined comparison and the predictive envelopes. Future work includes a 1T1R access-device model, measured or finite-difference in-situ gradients, broader retention-time distributions, fitting to raw per-point data beyond the digitized figures, and larger networks.

ACKNOWLEDGMENT

This work, including both the simulation experiments and the writing of this manuscript, was carried out with the assistance of Anthropic’s Claude (Claude Code). The author defined the research direction, made the key methodological choices, and verified all results. A detailed provenance log distinguishing human direction from AI-assisted execution is given in Appendix B.

REFERENCES

- [1] L. O. Chua, “Memristor: The missing circuit element,” *IEEE Trans. Circuit Theory*, vol. 18, no. 5, pp. 507–519, 1971.
- [2] D. B. Strukov, G. S. Snider, D. R. Stewart, and R. S. Williams, “The missing memristor found,” *Nature*, vol. 453, pp. 80–83, 2008.
- [3] D. Ielmini, “Resistive switching memories based on metal oxides: mechanisms, reliability and scaling,” *Semicond. Sci. Technol.*, vol. 31, no. 6, p. 063002, 2016.
- [4] D. Ielmini and H.-S. P. Wong, “In-memory computing with resistive switching devices,” *Nature Electronics*, vol. 1, pp. 333–343, 2018.
- [5] S. Yu, “Neuro-inspired computing with emerging nonvolatile memories,” *Proc. IEEE*, vol. 106, no. 2, pp. 260–285, 2018.
- [6] C. Yakopcic, T. M. Taha, G. Subramanyam, R. E. Pino, and S. Rogers, “A memristor device model,” *IEEE Electron Device Lett.*, vol. 32, no. 10, pp. 1436–1438, 2011.
- [7] A. Shafiee et al., “ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars,” in *Proc. ISCA*, 2016, pp. 14–26.
- [8] P. Chi et al., “PRIME: A novel processing-in-memory architecture for neural network computation in ReRAM-based main memory,” in *Proc. ISCA*, 2016, pp. 27–39.
- [9] M. Prezioso, F. Merrih-Bayat, B. D. Hoskins, G. C. Adam, K. K. Likharev, and D. B. Strukov, “Training and operation of an integrated neuromorphic network based on metal-oxide memristors,” *Nature*, vol. 521, pp. 61–64, 2015.
- [10] P. Yao et al., “Fully hardware-implemented memristor convolutional neural network,” *Nature*, vol. 577, pp. 641–646, 2020.
- [11] C. Li et al., “Analogue signal and image processing with large memristor crossbars,” *Nature Electronics*, vol. 1, pp. 52–59, 2018.
- [12] C. Li et al., “Efficient and self-adaptive in-situ learning in multilayer memristor neural networks,” *Nature Communications*, vol. 9, 2385, 2018.
- [13] Z. He, J. Lin, R. Ewetz, J.-S. Yuan, and D. Fan, “Noise injection adaption: End-to-end ReRAM crossbar non-ideal effect adaption for neural network mapping,” in *Proc. DAC*, 2019.
- [14] M. J. Rasch et al., “Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators,” *Nature Communications*, vol. 14, 5282, 2023.
- [15] V. Joshi et al., “Accurate deep neural network inference using computational phase-change memory,” *Nature Communications*, vol. 11, 2473, 2020.
- [16] B. K. Keshari, C. K. R. Madadi, S. DebRoy, A. Salimath, V. Mattela, and P. Sahatiya, “Self-rectifying AgO-based synaptic crossbar array for neuromorphic computing applications,” *Communications Materials*, 2026, doi: 10.1038/s43246-026-01165-2.

APPENDIX A

MODEL PARAMETERS AND IMPLEMENTATION NOTES

Table IV lists the default parameters used throughout. The array draws R_{on} and R_{off} from log-normal distributions (so resistances stay positive) and the thresholds from clipped Gaussians. The small-signal read conductance at V_{read} is $G_{\text{read}} = I(V_{\text{read}}, w)/V_{\text{read}}$, which is monotonic in w and used for all weight reads.

IR-drop solver. For an $N \times M$ array there are $2NM$ node voltages (a row-wire and a column-wire node per cross-point). Assembly is fully vectorized: device edges, row-wire edges, and column-wire edges are built as index arrays, the

TABLE IV
DEFAULT DEVICE, ARRAY, AND TRAINING PARAMETERS.

Symbol	Meaning	Value
R_{on}	LRS resistance (nominal)	2 k Ω , $\sigma=15\%$
R_{off}	HRS resistance (nominal)	200 k Ω , $\sigma=30\%$
V_{set}	SET threshold	0.80 V, $\sigma=0.08$
V_{reset}	RESET threshold	0.70 V, $\sigma=0.08$
k_s, k_r	switching rates	80
V_0	HRS nonlinearity scale	0.45 V
p	window exponent	4
V_{read}	read voltage	0.2 V
–	cycle-to-cycle noise	5%
$[g_{\text{lo}}, g_{\text{hi}}]$	weight window	[20, 300] μS
E_a, τ_0	retention (Arrhenius)	1.0 eV, 4.8×10^{-9} s
w_{eq}	retention equilibrium	0.02
H	hidden units (spiral MLP)	64
w_{max}	weight window (training)	10
–	batch size; float LR	32; 0.3
–	pulse LR; max pulse width	1.5×10^{-3} ; 3×10^{-3}
–	IR-train epochs; LR decay	350; 0.992
r_w	wire resistance (IR-train)	1 Ω/cell

diagonal is accumulated with `bincount`, and a single sparse matrix is factored with `scipy.sparse.linalg.splu`. The same factorization is reused for every input vector in a batch, which is what makes hardware-in-the-loop training affordable.

Differential decode. With input voltages $V_{\text{in}} = x V_{\text{read}}$ and a differential pair encoding W , the recovered pre-activation is $(I^+ - I^-)/(\text{decode scale} \cdot V_{\text{read}})$, which equals xW in the ideal case; this identity was used to verify the inference pipeline (ideal write reproduces the software accuracy in Table I).

APPENDIX B

SESSION PROVENANCE LOG

Following the project’s research protocol, we log what was human direction and what was AI-assisted execution. The research was conducted as an interactive session in which the human set the direction at each step and verified outcomes, and the AI assistant (Claude Code) wrote and debugged the simulation code, produced the figures, and drafted this manuscript. The work was not a pre-registered plan; it grew turn by turn, which is why the appendix records the actual order.

A. Human Direction (verbatim prompts, in order)

- 1) “explain how neuromorphic memristor works”
- 2) “lets say we’re using AlOx – i would like to visualize current-voltage graph and see a simulation of a single device”
- 3) “this is too much, i dont want animations, i want to scientifically understand how the I-V hysteresis happens and how it helps to retain memory state”
- 4) “can you convert this to a script that simulates device level behaviour and then create 64x64 crossbar array to see aggregate behaviour”
- 5) “add wire-resistance IR-drop via a proper nodal solve (this is where 64x64 vs 128x128 starts to matter), model

retention/drift as slow thermal relaxation of w , run a small YOLO inference layer on the array to quantify accuracy loss from variability”

- 6) “if YOLO is too large, can you pick another demonstrable learning task that can be shown to converge in this env itself, MNIST is too vanilla, also if required increase the crossbar dims”
- 7) “add retention drift between training and inference to see how fast a trained network degrades, and train with IR-drop in the forward pass to see whether the loop can learn around the wire resistance too”
- 8) “can you somehow visualize the crossbar array and the weights encoded and inference happening, like a test time inference”
- 9) “run the same animation through the IR-drop forward, or animate the retention drift”
- 10) “generate another viz where the two tiles are receiving signals (how the inputs are encoded to analog signals), and how the output is generated, visualize the waveforms over time”
- 11) “this viz isnt clear, can we make the efficiency takeaway much more apparent and clear”
- 12) “but i dont see the analog inputs coming into the crossbar array in parallel”
- 13) “treat this entire session as research, capture all the results and generate a detailed paper, use IEEE format”

B. Mapping of Prompts to Outputs

Table V maps each direction to the artifact the assistant produced and the section of this paper that reports it.

TABLE V
PROVENANCE: HUMAN DIRECTION TO AI-PRODUCED ARTIFACT.

Prompt	AI-produced artifact	Reported in
1	Conceptual explanation (no code)	–
2	Interactive device simulator (later set aside per prompt 3)	–
3	Written derivation of hysteresis and retention	Sec. III
4	<code>alox_crossbar.py</code> : device model, 64x64 array	Fig. 1–16
5	<code>crossbar_advanced.py</code> : MNA IR-drop, retention, conv + classifier	Fig. 3–6, Tab. II
6	<code>crossbar_learn.py</code> : in-situ two-spiral learning	Fig. 7
7	Post-training drift; IR-drop in training loop	Fig. 8
8–12	Visualizations of encoding, MVM parallelism, signal flow	App. C
13	This manuscript	–

C. Surprises and Debugging Insights

Several non-obvious issues were found and fixed during execution; they are logged here because they shaped the results.

- **Frequency collapse direction.** An early run showed the hysteresis loop growing rather than shrinking with frequency, because the chosen rates let the device fully switch at every tested frequency. Spacing the frequencies by decades exposed the correct monotonic collapse (Fig. 1c).
- **Inference decode bug.** The classifier first scored at chance despite an ideal write; the cause was an input-voltage scale factor left out of the differential decode while the bias was not scaled. Cosine similarity hid the bug in the convolution check because it is scale invariant.
- **Weight-range clipping.** In-situ learning stalled near 80 percent until the conductance window was widened to encode weights up to about ± 8 ; the spiral network needs larger weights than a ± 3 window allows.
- **Noise as dither.** Noiseless identical devices stalled under deterministic nonlinear updates, while cycle-to-cycle noise enabled convergence.
- **Surrogate-gradient noise.** Hardware-in-the-loop IR-drop training oscillated because the backward pass uses an ideal-weight surrogate; a pulse-size decay schedule settled it (Fig. 8c).

D. Division of Labour

Human contribution: the overall direction (start from device physics, scale to an array, then add each non-ideality), the specific methodological asks (a proper nodal solve, drift between training and inference, IR drop inside the training loop, a non-trivial learning task), the decision to set aside an interactive demo in favour of a reproducible script, and verification of each result. AI contribution: implementation of the device and array models, the vectorized MNA solver, the program-verify and in-situ training loops, all figures, the debugging listed above, and the drafting of this paper. All numerical results were produced by the released code and inspected by the human before inclusion.

APPENDIX C ADDITIONAL VISUALIZATIONS

Figure 16 (referenced from Section VI-C) shows the program-and-read of an image into the array and the analog MVM scatter. Figures 17 and 18 are stills from animations built to communicate the in-memory parallelism: the first contrasts the $N \times M$ sequential multiply-accumulate of a von Neumann engine with the single parallel read of the crossbar; the second shows the analog inputs entering all wordlines simultaneously and current summing down all bitlines at once.

APPENDIX D REPRODUCIBILITY

All code accompanying this paper is in the public GitHub repository <https://github.com/rayjyotir5/alox-crossbar-nonidealities> (Python 3.11; NumPy, SciPy, scikit-learn, PyTorch, Matplotlib). It consists of three core modules and several scripts:

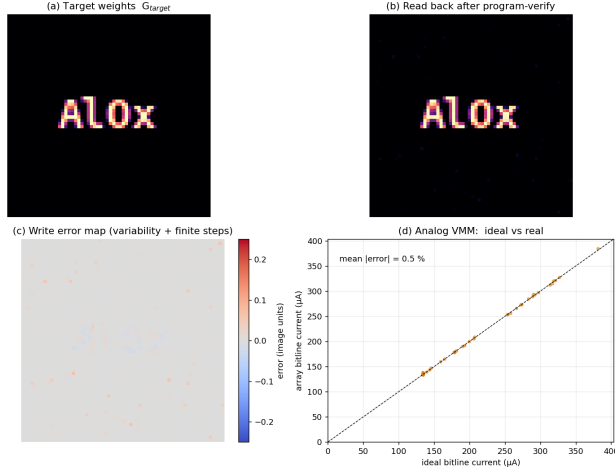


Fig. 16. Crossbar as compute. (a) Target weights (an image) and (b) the array read-back after program-verify. (c) Write-error map. (d) Analog MVM versus ideal, about 0.5 percent mean error.

- `alox_crossbar.py`: device model, single-device sweep, 64x64 array variability, image program/read, analog MVM (Fig. 1–16).
- `crossbar_advanced.py`: vectorized MNA IR-drop solver, Arrhenius retention, convolutional-layer fidelity, and the digit-classifier accuracy study (Fig. 3–6, Tab. II).
- `crossbar_learn.py`: in-situ two-spiral training, hardware-in-the-loop IR-drop training, and post-training retention (Fig. 7–8).
- `seeds_experiments.py` and `harden2.py`: the multi-seed study (classifier, in-situ on spirals and circles), the mixed-state IR-drop table, the Arrhenius E_a sensitivity, and the output-calibration baseline for the IR-drop recovery. All headline numbers in the paper are produced here as $\text{mean} \pm \text{std}$ over seeds.
- `keshari_phase_energy.py`: calibration to the published Ag_2O device, the MNIST validation, the operating-envelope phase diagrams, and the energy model.
- `aihwkit_baseline.py` (run in an AIHWKit virtual environment): trains the MNIST MLP and produces the AIHWKit analog-inference baseline and the shared weights.
- Visualization scripts (`crossbar_*_viz.py`) produce the animations whose stills appear in App. C.

Each module is self-contained and writes its figures on execution; the full pipeline runs in minutes on a single laptop CPU. Each headline result is reported over multiple seeds (typically five for fast experiments, three for the slower IR-aware training runs); the figures show a single representative seed while the tables and text report the seed statistics.

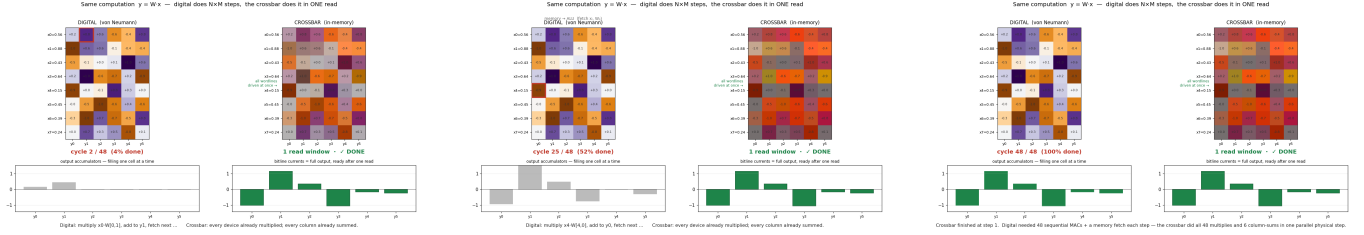


Fig. 17. The same MVM done two ways, shown at three times (left to right). The digital engine steps through $N \times M$ multiply-accumulates with a memory fetch each cycle (its counter climbs 1, mid, $N \times M$); the crossbar finishes the whole product in one parallel read at step 1 and then holds the answer.

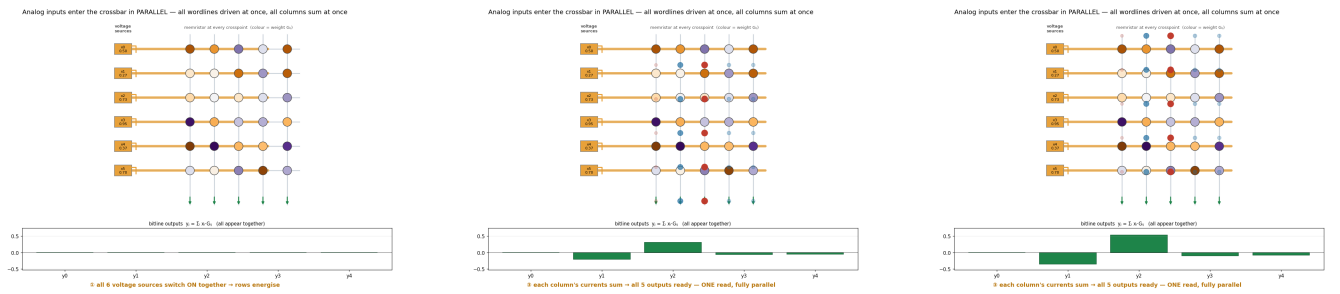


Fig. 18. Analog inputs entering the crossbar in parallel, shown at three times (left to right): all wordlines are driven at once, every device passes $I = VG$, and current then sums down every bitline simultaneously into the outputs.